iregi-rust benchmark

Batched one-loop tensor reduction in Rust versus IREGI and MadLoop

2026-10-23 · Timing environment not yet recorded

Reading guide. Every timing ratio is *iregi-rust time ÷ reference time*. Green means faster, white parity and red slower; timings are per phase-space point. Gray cells are unmeasured and amber cells are pending verification. Source inspection does not establish numerical agreement. Report infrastructure and source-convention audit are available. No performance measurements or reference agreement are claimed by this data file.

Acceptance gates

Correctness	Agreement with original IREGI and isolated MadLoop graphs	**Pending**
Pure reduction	Equal or better performance on identical inputs	**Unmeasured**
Complete evaluation	Report preparation, numerator, reduction and masters separately	**Unmeasured**

1 Identical-input comparisons

Table 1: **Synthetic inputs versus original IREGI.** Each measured comparison must share routing, masses, scale, rank, numerator coefficients and reduction mode. The current cells contain no measurements.

Case	n / r	Reduction	Masters	Total
Bubble Explicit DOT numerator	2 / 0–3	**Unmeasured** vs IREGI	**Unmeasured** vs OneLOop	**Unmeasured** vs IREGI
Triangle Explicit DOT numerator	3 / 0–3	**Unmeasured** vs IREGI	**Unmeasured** vs OneLOop	**Unmeasured** vs IREGI
Box Expanded topology coverage	4 / planned	**Unmeasured** vs IREGI	**Unmeasured** vs OneLOop	**Unmeasured** vs IREGI



2 Physical diagram comparisons

Pending. FeynKit-generated physical numerators and graph-isolated MadLoop comparisons have not yet been measured. R2 and UV counterterms are excluded; bare UV/IR poles and R1 remain. A physical graph comparison must identify its model, routing, normalization, helicity and color configuration.

3 Validation coverage

Table 2: **Evidence ledger.** "Source audited" records code inspection only. Every other capability still requires executable evidence.

Capability	Required evidence	Status
Coefficient ordering and normalization	Native IREGI and MadLoop conventions Original Fortran interfaces and scalar wrappers inspected; numerical checks pending.	**Source audited**
Scalar-master backend	OneLoopMaster, all batch lanes Native evaluation integration and reference comparison pending.	**Pending**
Precision	binary64, DoubleFloat, arbitrary precision No numerical coverage claimed.	**Pending**
Automatic differentiation	Dual values and derivatives Derivative checks pending.	**Pending**
Vectorization	Batches propagated through masters Widths 1, 4, 8, 16, 32 and 64 planned.	**Pending**
Exceptional kinematics	Lane-local errors and precision rescue Threshold and small-Gram corpus pending.	**Pending**
Physical verification	Isolated MadLoop graph, no R2 / UV counterterms Bare poles and R1 must remain in the comparison.	**Pending**

4 Measurement protocol

Run release builds on one pinned CPU core with fixed thread counts and record hardware, operating system, toolchain, compile flags, dependency revisions, input hashes, seeds, warm-up, sample count and raw observations. Every result must identify whether caches are cold or warm. Report median and dispersion over repeated sufficiently long timed blocks; retain an output checksum.

Preparation comprises graph import, rules, tensor identities, coefficient extraction and evaluator construction. Time numerator evaluation, pure reduction, scalar masters and complete evaluation independently. Measure the complete total directly: adding separately instrumented components is only a diagnostic. Distinguish reference-library calls from subprocess and file I/O.

Use the same samples at batch widths 1, 4, 8, 16, 32 and 64 and report time per point. For precision comparisons, match the numerical precision and include rescue costs. Record failed lanes; do not omit unfavorable points silently. Absolute/scaled coefficient errors, high-precision adjudication and derivative checks accompany performance results. No speed gate passes without correctness.

5 Rebuilding this report

Update `docs/data/results.json` only from retained measurements and rebuild with `typst compile docs/benchmark.typ docs/benchmark.pdf`. The report uses Libertinus Serif and DejaVu Sans/Mono. Supply their installation directories through Typst's `--font-path` option when needed.

6 Reference conventions

These conventions were read from the supplied original sources. Their presence in this report is not a claim that every adapter has passed its numerical tests.

6.1 Kinematics and scale

Use Minkowski signature $(+,-,-,-)$ with component order (p^0, p^1, p^2, p^3). Propagators are associated with loop shifts and **squared** internal masses. Native tensor reduction accepts one shift per propagator and removes the first common shift internally, then translates tensor coefficients back. The MadLoop interface instead receives the nonzero shifts and inserts a zero first shift. A reference comparison must preserve the chosen numerator routing when it changes the propagator direction.

The original `MU` / `MUR` argument is an energy scale, not its square. Its scalar wrapper passes μ^2 to QCDLoop and μ to OneLOop. Complex mass-squared inputs must retain their imaginary part; the reference switches to its OneLOop backend for complex masses.

6.2 Native poles and MadLoop conversion

The native result order is

$$(U, I, J, F) = (\varepsilon_{\mathrm{UV}}^{-1}, \varepsilon_{\mathrm{IR}}^{-1}, \varepsilon_{\mathrm{IR}}^{-2}, \mathrm{finite})$$

. These are separate coefficients. In particular, a scaleless integral can have canceling UV and IR contributions even when its combined Laurent series is zero. Three combined coefficients do not determine their separate origin. OneLoopMaster uses $(\mathrm{finite}, \varepsilon^{-1}, \varepsilon^{-2})$ ordering.

The original `IMLOOP` adapter combines the native poles and maps a tensor of homogeneous rank r to MadLoop order (finite, simple pole, double pole). Defining $a = \gamma_E + \ln \pi$, $b = \frac{a^2}{2} + \frac{\pi^2}{12}$ and $c_r = \frac{(-1)^r}{i\pi^2}$, its explicit mapping is

$$(F_{\mathrm{ML}}, S_{\mathrm{ML}}, D_{\mathrm{ML}}) = c_r(F + a(U + I) + bJ, U + I + aJ, J).$$

This formula describes the reference's native-to-MadLoop conversion; applying it to an already normalized master result would count normalization twice. Validate the native master normalization before connecting these layers.

6.3 Polynomial and tensor interfaces

A rank bound R has $\binom{R+4}{4}$ Cartesian coefficients in the original four-component basis. The precise component order comes from `funlib::calc_pos` and must be shared by numerator preparation and tensor output. The original native and MadLoop tensor conventions differ in multinomial factors: the native path multiplies symmetric components by the factor returned by `all_integers`, whereas `ML5_CONVENTION=.TRUE.` uses the unmultiplied components. Test mixed monomials explicitly; diagonal-only tests cannot detect this mismatch.

Reduction modes are 0: IBP, 1: PaVe, 2: PaVe with IBP stability improvement. Direction mode 2 reverses the propagator order and shifts the loop origin. The source labels direction modes 4/5 as unimplemented quadruple precision; their numbers are not evidence of high-precision support.

6.4 Physical comparisons

Remove R2 and UV counterterms from both physical pipelines while retaining bare poles and R1. Compare the same isolated graph with matched parameters, polarizations, helicity/color factors and overall normalization. Individual graphs depend on gauge and conventions; an agreement claim must document those choices rather than compare an unspecified process sum.

Source locations: `references/madgraph/vendor/IREGI/src/IREGI4ML5_interface.f90`, `ti_reduce.f90`, `si_reduce.f90`, `mis_warp.f90`, `funlib.f90` `references/oneloopmaster/src/lib.rs`.

7 Algebra reuse audit

All symbolic algebra belongs to Symbolica and the FeynKit / Spenso / Idenso ecosystem. Before introducing a missing operation, record three checks: Symbolica APIs and implementation; Symbolica usage/examples/tests; and FeynKit/gammaloop facilities and tests. An adapter or data-layout conversion must not grow an independent polynomial, simplification or matrix engine.

7.1 Existing facilities to reuse

- **Tensor identities and contractions:** Idenso contracts metrics and reduces supported gamma/color identities; Spenso carries tensor structure. Keep Lorentz dimension and any gamma-five prescription explicit.
- **Coefficient extraction:** FeynKit's tensor reducer uses Idenso contraction without global expansion, `to_dots()` and the shared `coefficient_list` collector. Preserve scalar factorization until polynomial extraction requires expansion.
- **Propagator and scalar-product algebra:** FeynKit's `IntegralFamily` uses Symbolica for polynomial conversion, exact rank and linear solves, basis completion and mappings. Its Gaussian-rational conversion avoids statistical expression-field zero tests that may discard small coefficients.
- **Numerical preparation:** Compile reusable Symbolica expressions outside the sample loop. Evaluation and scalar-master types must preserve the selected precision and all Dual components without binary64 round trips.

7.2 HEPKit tensor-reduction optimizations

The local `feynkit-tensor` reducer stores repeated-vector contractions as symmetry orbits instead of factorially many labeled pairings. Its joint-color dynamic program builds orbit intersections; alternating cycles build orbit Gram matrices; Symbolica solves the smaller orbit system. A rank-20 isotropic projection can have one orbit instead of 19!! labeled pairings.

The implementation caches canonical Weingarten coefficients and has a closed isotropic path. General projectors are validated through rank 20; the special all-equal isotropic route reaches rank 32. These are upstream limits, not a statement of iregi-rust's current supported domain.

For denominators with external momenta, declare an independent external basis and average only the transverse space. Odd total tensor ranks need not vanish. Keep D symbolic while constructing high-rank projectors; early integer specialization can make the universal metric basis singular. External Gram inverses can amplify small-Gram numerical errors, so projector preprocessing requires a conditioning-aware comparison against direct recurrences.

This projector reduces tensor numerators, not complete scalar integral families. Reuse it where applicable and benchmark the resulting plan; no factorial-count argument alone establishes a numerical speedup. Retain explicit output/pairing limits and report their failures honestly.

7.3 References and provenance

Alwall et al., appendix C.1: original IREGI context.
HEPKit TensorReducer API.
Goode et al.: symmetry-orbit tensor projectors.
Collins and Matsumoto: Weingarten orthogonality recurrence.
NinjaRS benchmark: report typography, table and color-style reference.

Local audit targets: `references/gammaloop_feynkit/crates/feynkit-tensor/src/` (`reduction.rs`, `reduction/external.rs`, `orbit_projector.rs`, `weingarten.rs`); `crates/feynkit-graph/src/integrals.rs`.